



Nombre: _____ DNI: _____

Apellidos: _____ Grupo: _____

Cuestión 1 Considere que un sistema de ficheros basado en i-nodos contiene la siguiente información:

Mapa de bits: 1 0 0 1 0 1 1 1 0 0 0 0 1 0 0 1 0 0 . . . 0

I-nodos:	i-nodo 2		i-nodo 3		i-nodo 4		i-nodo 5		i-nodo 9	
	Tamaño	1	Tamaño	2	Tamaño	1	Tamaño	1	Tamaño	1
	Enlaces	N/A	Enlaces	1	Enlaces	1	Enlaces	N/A	Enlaces	N/A
	Tipo	D	Tipo	F	Tipo	F	Tipo	D	Tipo	D
	Permisos	rwX	Permisos	rw-	Permisos	rw-	Permisos	r-x	Permisos	rwX
	Directo	3	Directo	6	Directo	12	Directo	0	Directo	5
		Indirecto			Indirecto			Indirecto		
		NULL			NULL			NULL		

Bloque 0		Bloque 3		Bloque 5		Bloque 6		Bloque 7		Bloque 12		Bloque 15	
.	5	.	2	.	9	AAAA		15		BBBB		CCCC	
..	2	..	2	..	5								
C	9	A	3										
D	4	B	5										

a) Dibuje un diagrama de la jerarquía de ficheros y directorios descrita mediante las anteriores estructuras. Utilice rectángulos para los directorios y círculos para los ficheros. En el caso de ficheros regulares, especifique su contenido.

b) Indique cuáles de las anteriores estructuras se modificarían, y de qué manera, al ejecutar cada una de las siguientes órdenes de forma sucesiva (si alguna de ellas devuelve un error, indícalo):

1. cp /A /B
2. mv /B/D /
3. truncate -s 0 /B/A

c) A partir de las estructuras originales, detalle cómo se organizaría este mismo contenido bajo un sistema FAT32. Concretamente, muestre el contenido de:

1. La tabla FAT (en sus 16 primeras entradas).
2. El directorio raíz, con todos los campos de los que disponga de información.
3. Los bloques de datos (en la medida de lo posible, utilice los mismos bloques de datos que los usados en el sistema basado en i-nodos).



Cuestión 2 En la cafetería de cierta facultad se sirven bocadillos de tres tipos: lomo con queso (tipo 0), calamares (tipo 1) y tortilla (tipo 2). Cuando un estudiante desee comer un bocadillo de un cierto tipo ha de acercarse a la barra a recogerlo, visita que puede ser utilizada para retirar también otros bocadillos del mismo tipo para sus compañeros. Si cuando el estudiante se acerca a la barra no queda ningún bocadillo del tipo elegido, deberá avisar al camarero, que podría estar bloqueado en ese momento. Cuando el camarero esté despierto comprobará qué tipo de bocadillos hay que reponer –aquellos para los que haya menos de K unidades–, repondrá N unidades del tipo o tipos que falten –invocando `reponerBocadillos(tipoBocadillos, N)`– y se bloqueará. Nótese que K y N son constantes, con $0 < K \leq N$.

Una vez haya bocadillos del tipo elegido, el estudiante se servirá uno y retirará todos los que pueda llevar a sus compañeros (hasta un máximo de $m-1$), invocando la función `retirarBocadillos(tipo, número)`, que emula dicha acción. En cuanto un estudiante consiga al menos un bocadillo, podrá comer, y también repartir el resto de bocadillos obtenidos entre sus compañeros.

Un número arbitrario de estudiantes y el camarero (hilos del programa concurrente) se comportan del siguiente modo:

```
#define N ...
#define K ...

void Estudiante(int tipoBocadillo, int m){
    while (true) {
        int bocadillosObtenidos=conseguirBocadillos(tipoBocadillo,m);
        repartirBocadillos(bocadillosObtenidos-1);
        comer();
    }
}

void Camarero(){
    while (true) {
        servirBocadillos();
    }
}
```

Implemente las funciones `conseguirBocadillos()` y `servirBocadillos()` empleando mecanismos de sincronización (mútexes, variables condición y/o semáforos) y otras variables compartidas.

Se ha de tener en cuenta que sólo es seguro invocar `retirarBocadillos(tipoBocadillo, n)` si quedan al menos n bocadillos del tipo correspondiente. Análogamente, sólo se ha de invocar `reponerBocadillos(tipoBocadillo, N)` desde `servirBocadillos()` si quedan menos de K bocadillos de ese tipo.



Cuestión 3 Considere un sistema monoprocesador en el que el sistema operativo aplica una determinada política de planificación de tipo expropiativo. Inicialmente, hay 3 procesos independientes en la cola del planificador: P1, P2 y P3 (en este orden). Estos procesos se ejecutan de forma indefinida siguiendo los siguientes patrones de CPU y E/S:

P1 (3-CPU, 5-E/S), P2 (5-CPU, 3-E/S) y P3 (4-CPU, 4-E/S)

El siguiente diagrama muestra el estado de los procesos durante las primeras 20 unidades de tiempo en el que se representa con “X” cuando los procesos están en ejecución, con “–” cuando están bloqueados y con “O” cuando están preparados para ejecución.

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
P1	X	X	O	O	O	O	X	–	–	–	–	–	X	X	X	–	–	–	–	–
P2	O	O	X	X	O	O	O	X	X	O	O	X	–	–	–	X	X	O	O	X
P3	O	O	O	O	X	X	O	O	O	X	X	–	–	–	–	O	O	X	X	O

- a) ¿Puedes identificar un tipo de política de planificación de los que se han estudiado en la asignatura que es compatible con este diagrama de estados? Justificar la respuesta.

- b) En caso de respuesta afirmativa en la cuestión anterior ¿Necesita el planificador algún recurso hardware específico para poder implementarse dicha política? Justificar la respuesta.

- c) Si se trata de un sistema de propósito general, ¿Cuál es el valor más razonable para la unidad de tiempo del diagrama: 4 ns, 4ms ó 4s? Justifica la respuesta.

Cuestión 4 Un sistema utiliza gestión de memoria virtual con paginación bajo demanda con páginas de 4KiB, direcciones de memoria de 64 bits y memoria principal de 16GB. En dicho sistema se desea ejecutar el siguiente programa:

```
1 #include <lcool.h>
2 #include <stdio.h>
3 /* resto de includes */
4 ...
5 #define M 900
6
7 void *get_cool_numbers(void *arg)
8 {
9     int *p, i, n;
10    n = (int) arg;
11    p = (int *) malloc(n * sizeof(int));
12    if (!p)
13        return NULL;
14    for (i = 0; i < n; i++)
15        p[i] = lcool_getnum();
16    //C
17    return (void *) p;
18 }
19
20 int main(int argc, char *argv[])
21 {
22     int pid;
23     // A
24     pid = fork();
25     // B
26     if (pid == 0) {
27         pthread_t tid[2];
28         int *p1, *p2, i;
29         pthread_create(&tid[0], NULL, get_cool_numbers, (void *) M);
30         pthread_create(&tid[1], NULL, get_cool_numbers, (void *) M);
31         pthread_join(tid[0], (void *)&p1);
32         pthread_join(tid[1], (void *)&p2);
33         //D
34         for (i = 0; i < M; i++) {
35             int sum = p1[i] + p2[i];
36             if (lcool_iscool(sum))
37                 printf("Cool addition: %d + %d = %d is still cool\n",
38                     p1[i], p2[i], sum);
39             else
40                 printf("Cool addition: %d + %d = %d is not cool anymore\n",
41                     p1[i], p2[i], sum);
42         }
43     } else if (pid > 0) {
44         while (wait() > 0);
45         //E
46         printf("That was cool, wasn't it!\n");
47     }
48     return 0;
49 }
```

Assumiendo que:

- La sección `.text` del binario ocupa 4000 bytes, enlazada estáticamente con la biblioteca estándar del sistema (incluida en los 4000 bytes).
- La sección `.data` ocupa 16 bytes (biblioteca estándar) y no hay sección `.bss`.
- La biblioteca `lcool` sólo tiene región de código que ocupa 2200 bytes, y se enlaza dinámicamente con el programa.
- El contenido inicial de la pila al lanzar a ejecución el programa ocupa 16 bytes.
- Al comenzar la ejecución de la función `main` no hay región `heap`.
- Que un entero (`int`) ocupa 4 bytes.

Responda a las siguientes preguntas:



- a) Identifique qué regiones lógicas (y su tamaño en páginas) constituyen la imagen de memoria de los procesos en los puntos marcados en el código como A, B, C, D y E

- b) Asumiendo que hay marcos libres suficientes para el proceso, identifica las líneas de la función `get_cool_numbers` que provocarán un fallo de página en la primera ejecución de esta función, explicando el motivo y la región de código afectada.

Cuestión 5 Un proceso en UNIX ejecuta el siguiente código (run_processes):

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <sys/types.h>
5 #include <sys/wait.h>
6
7 int main(int argc, char *argv[]) {
8     int value, status;
9     pid_t pid;
10
11     value = atoi(argv[1]);
12     if (value <= 0)
13         return 0;
14
15     pid = fork();
16     if (pid == 0) {
17         fork();
18         value = value - 1;
19         sprintf(argv[1], "%d", value);
20         execvp(argv[0], argv);
21         perror("`exec");
22         exit(1);
23     } else {
24         while (wait(&status) != pid)
25             continue;
26     }
27     return 0;
28 }
```

Asumiendo que:

- La función *atoi()* devuelve el valor entero representado por la cadena de caracteres dada.
- Dado un valor entero y un buffer, *sprintf()* almacena en el buffer la tira de caracteres que representa el entero.
- Se supone que los servicios *fork()* y *execvp()* se ejecutan de forma correcta.

Responda a las siguientes preguntas:

a) ¿Cuál sería el número total de procesos creados durante la ejecución de run_processes 2, sin contar el proceso padre original?

b) Dibujar el árbol de procesos resultantes de la ejecución de run_processes 2.

c) Razonar si al realizar la ejecución de run_processes 2, podrá existir algún proceso huérfano y/o algún proceso zombie. Se considera proceso huérfano aquel cuyo padre ha muerto, y es adoptado por el proceso *Init*.